

## Lecture 34 — Embedded Real-Time Operating Systems

Jeff Zarnett

2026-09-01

## Time, Yes, Time Indeed

The course introduction already introduced the premise of real-time operating systems, and the discussion of real-time scheduling algorithms made it abundantly clear that a typical desktop operating system is not a suitable choice for a real-time system. It is also common for the real-time systems running the most life-and-safety critical things to be embedded; that is, they run on microcontrollers or tiny microprocessors. There's accordingly no capacity on the system to run all the extra parts of the operating system that are not needed. The scheduling topic also said that a real-time system lives and dies on scheduling. That's true, largely, because the wrong scheduling choices guarantee failure. Meeting deadlines in a real-time system, though, is a weakest-link kind of problem: if any one of the things to be done takes "too long", the deadline is missed.

To get specific, we are going to discuss *FreeRTOS*, which is exactly what it says on the label, a free real-time operating system; the project describes itself as a collection of libraries that includes a kernel and the necessary additional functionality [Bar24]. For anything but a system that does exactly one thing at all times, we likely need very little motivation for why we should use an operating system rather than just run everything without one. But if you need some: no need to rewrite things like memory management, scheduling, drivers, or power management. You know... just the things we talked about in the course so far.

## References

- [Bar24] Richard Barry. *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Amazon Web Services, 1.1.0 edition, 2024. Referencing FreeRTOS kernel V10.6.2. Available at <https://github.com/FreeRTOS/FreeRTOS-Kernel-Book>.